TrustSense-BMS

Software Requirements Specification

IEEE Standard: IEEE Std 830-1998

Date: August 2026

Status: In-Progress

Project Team

Yash Borade | Medha Karadiguddi | Sai Subhransu Dash | Krushna Sonar

Industry Sponsor

VG Marine Tech Pvt. Ltd.

Academic Guide

Prof. Pallavi Yevale

Table of Contents

1. Introduction 1

1.1 Purpose 1

1.2 Document Conventions 1

1.3 Intended Audience and Reading Suggestions 1

1.4 Project Scope 1

1.5 References 2

2. Overall Description 3

2.1 Product Perspective 3

2.2 Product Features 3

2.3 User Classes and Characteristics 3

2.4 Operating Environment 3

2.5 Design and Implementation Constraints 3

2.6 User Documentation 4

2.7 Assumptions and Dependencies 4

3. System Features 5

3.1 AI Virtual Sensing 5

3.1.1 Description and Priority 5

3.1.2 Stimulus / Response Sequences 5

3.1.3 Functional Requirements 5

3.2 Cyber Intrusion Detection System (IDS) 5

3.2.1 Description and Priority 5

3.2.2 Stimulus / Response Sequences 5

3.2.3 Functional Requirements 5

3.3 Blockchain Battery Passport 6

3.3.1 Description and Priority 6

3.3.2 Stimulus / Response Sequences 6

3.3.3 Functional Requirements 6

3.4 Engineering RAG Assistant 6

3.4.1 Description and Priority 6

3.4.2 Stimulus / Response Sequences 6

3.4.3 Functional Requirements 6

4. External Interface Requirements 7

4.1 User Interfaces 7

4.2 Hardware Interfaces 7

4.3 Software Interfaces 7

4.4 Communications Interfaces 7

5. Other Nonfunctional Requirements..... 8
5.1 Performance Requirements..... 8
5.2 Safety Requirements..... 8
5.3 Security Requirements..... 8
5.4 Software Quality Attributes..... 8
6. Other Requirements..... 9
6.1 Logical Database Requirements..... 9
6.2 Regulatory Compliance..... 9
6.3 Risk Management (FMEA)..... 9
Appendix A: Glossary..... 10
Appendix B: Analysis Models..... 11
B.1 System Sequence Diagram..... 11
B.2 Entity Relationship Diagram (ERD)..... 11
Appendix C: Open Issues List..... 12

1. Introduction

1.1 Purpose

The purpose of this document is to define the complete software requirements for the TrustSense-BMS project. It outlines the functional and non-functional requirements, external interfaces, structural models, and design constraints for an integrated AI, Cybersecurity, and Blockchain Battery Management platform. This document serves as a binding contract and blueprint for the internal development team, the industry sponsor at VG Marinetech (Prasun Sir), and the academic guide, Prof. Pallavi Yevale.

1.2 Document Conventions

- **Priorities:** Functional requirements (REQ) are prioritised as High, Medium, or Low.
- **Format:** All functional requirements use the `REQ-X.Y` numbering scheme.
- **TBD:** "To Be Determined" — information pending final hardware procurement.

1.3 Intended Audience and Reading Suggestions

This document is intended for three audiences: **Developers** (to build the system), **Project Managers** (to track scope), and **Compliance Auditors** (to verify blockchain requirements). Readers should begin with Section 2 for an overall system overview before proceeding to the deep technical feature specifications in Section 3.

1.4 Project Scope

TrustSense-BMS is a "Physics-Informed AI and Blockchain Integrated Battery Management System." Modern Electric Vehicles (EVs) suffer from fragile physical thermal sensors, vulnerable Controller Area Networks (CAN), and untraceable battery life histories.

This software acts as an intelligence overlay that achieves three primary objectives:

1. Reduces physical sensor dependency by >50% using a Graph Neural Network (GNN).
2. Secures the vehicle's OT network against data spoofing using a Time-Series Transformer Intrusion Detection System (IDS).
3. Creates an immutable, cryptographically secure Battery Passport via Hyperledger Fabric to comply with EU mandates.

1.5 References

- *Temperature prediction by graph convolutional networks* (2024)
- *Time series anomaly detection in vehicle sensors using self-attention mechanisms*, IEEE Transactions (2024)
- *EU battery regulation and the role of blockchain in creating digital battery passports* (2024)
- *Integrating Physics-Based Models with Machine Learning for Battery State Estimation* (2023)
- *CAN Bus Spoofing and Cyber-Physical Validations* (2025)

2. Overall Description

2.1 Product Perspective

TrustSense-BMS operates as a secondary, advisory intelligence layer. It sits alongside, but remains logically separate from, the primary hardware BMS. The system ingests raw CAN bus telemetry (read-only), processes it through local Python-based AI models, and visualises results on a React dashboard, while simultaneously logging cryptographic hashes to a local blockchain network.

2.2 Product Features

- **AI Virtual Sensing:** Mathematically reconstructs internal cell temperatures without physical sensors.
- **Cyber IDS:** Detects anomalous, thermodynamically impossible data packets in real time.
- **Blockchain Battery Passport:** Generates an immutable, cryptographic history of the battery pack.
- **Engineering RAG Assistant:** Provides a local LLM interface to query battery history using natural language.

2.3 User Classes and Characteristics

- **Battery Engineers / Analysts:** Technically proficient users requiring deep insights into thermal states. High frequency of use.
- **Compliance Auditors:** Non-technical, regulatory users requiring simple cryptographic proofs. Low frequency of use.

2.4 Operating Environment

- **Hardware Platform:** NVIDIA Jetson Orin Nano (Edge AI Node).
- **Operating System:** Ubuntu 22.04 LTS (Linux).
- **Network:** Local isolated intranet (for security).

2.5 Design and Implementation Constraints

- **GPU Restraints:** The Llama 3 LLM requires significant VRAM, constraining concurrent inferences on the Jetson Orin Nano.
- **Budget Restraints:** The system must utilise 100% open-source frameworks — React, FastAPI, PyTorch, Hyperledger — to ensure zero licensing costs.

2.6 User Documentation

- An interactive PDF Engineering Manual covering system architecture, API endpoints, and deployment.
- A `README.md` containing Docker Compose deployment instructions for all four microservices.

2.7 Assumptions and Dependencies

- **Assumption:** The physical testbench provides standard OBD-II/CAN data streams in real-time at 10 Hz or greater.
- **Dependency:** The system strictly depends on the PyBaMM library to accurately generate synthetic physical boundaries and thermodynamic equations for the GNN training process.

3. System Features

3.1 AI Virtual Sensing

3.1.1 Description and Priority

Computes the internal temperatures of battery cells that lack physical sensors using a Graph Neural Network (GNN). **Priority: High.**

3.1.2 Stimulus / Response Sequences

- **Stimulus:** Ingests raw voltage (V) and current (I) telemetry every 100 ms from the CAN bus adapter.
- **Response:** Broadcasts the inferred temperature (°C) of all unsensed nodes to the WebSocket stream.

3.1.3 Functional Requirements

- `REQ-1.1:` The GNN module shall compute estimated temperatures using a spatial adjacency matrix derived from the cell topology.
- `REQ-1.2:` The system shall flag a node as `HOTSPOT` if the inferred temperature exceeds 45°C.

3.2 Cyber Intrusion Detection System (IDS)

3.2.1 Description and Priority

Analyses CAN bus traffic using Time-Series Transformers to detect data spoofing and cyberattacks in real time. **Priority: High.**

3.2.2 Stimulus / Response Sequences

- **Stimulus:** A CAN payload byte stream arrives at the IDS module.
- **Response:** Flags the packet as `SAFE` or `SPOOFED_DATA` based on physics-informed boundary validation.

3.2.3 Functional Requirements

- `REQ-2.1:` The IDS shall calculate the Shannon entropy of payload bytes over a rolling 1-second window.
- `REQ-2.2:` The physics-validator shall flag any packet indicating a temperature delta > 5°C per second as `SPOOFED_DATA`.

3.3 Blockchain Battery Passport

3.3.1 Description and Priority

Anchors a cryptographic hash of the battery's telemetry batch to a distributed Hyperledger Fabric ledger every 60 seconds. **Priority: Medium.**

3.3.2 Stimulus / Response Sequences

- **Stimulus:** 60 seconds of telemetry is aggregated off-chain into a JSON batch.
- **Response:** A SHA-256 hash of the batch is successfully written to the Hyperledger chaincode and returned with a transaction ID.

3.3.3 Functional Requirements

- `REQ-3.1:` The system shall compute a SHA-256 hash of the JSON telemetry batch every 60 seconds.
- `REQ-3.2:` The system shall execute a Hyperledger Fabric `PutState` transaction to store the hash immutably on the ledger.

3.4 Engineering RAG Assistant

3.4.1 Description and Priority

A conversational LLM interface that allows engineers to query the battery's historical telemetry using natural language, powered by Retrieval-Augmented Generation (RAG). **Priority: Low.**

3.4.2 Stimulus / Response Sequences

- **Stimulus:** An engineer submits a natural language question via the React dashboard chat interface.
- **Response:** The Llama 3 LLM returns a grounded answer using context documents retrieved from ChromaDB.

3.4.3 Functional Requirements

- `REQ-4.1:` The Llama 3 LLM shall generate responses using *only* context retrieved from the local ChromaDB vector store, preventing hallucinations.

4. External Interface Requirements

4.1 User Interfaces

- A React-based web dashboard featuring a 3D interactive heatmap of the battery pack with per-cell colour coding.
- An Anomaly Score ticker (0–100%) showing the current cyber threat level in real time.

4.2 Hardware Interfaces

- Interface via USB or GPIO to a standard CAN bus adapter (e.g., PCAN-USB or SocketCAN) to read 11-bit and 29-bit CAN frames at standard baud rates.

4.3 Software Interfaces

- **PostgreSQL / TimescaleDB:** High-speed hypertable storage of raw, off-chain telemetry.
- **Hyperledger Fabric API:** Chaincode `PutState` and `GetState` transactions for the Battery Passport.
- **ChromaDB:** Local vector database for RAG embedding storage and similarity search.

4.4 Communications Interfaces

- **RESTful HTTP** (`GET /api/history`, `GET /api/passport/{id}`) for historical data loading.
- **WebSockets** (`ws://localhost:8000/stream`) for low-latency real-time telemetry and alert streaming.

5. Other Nonfunctional Requirements

5.1 Performance Requirements

- The GNN Virtual Sensor shall maintain a thermal estimation error of RMSE ≤ 1.0°C under nominal operating conditions.
- The Cyber IDS shall classify an incoming CAN packet as safe or spoofed within ≤ 500 ms of receipt.

5.2 Safety Requirements

- The system **SHALL NOT** issue physical control commands (e.g., open contactors, limit current) to the high-voltage battery hardware. It acts strictly as a passive monitoring and advisory platform.

5.3 Security Requirements

- Raw proprietary telemetry must **never** be transmitted to external cloud APIs (e.g., OpenAI). All LLM inference shall execute locally on the Jetson Orin Nano.

5.4 Software Quality Attributes

- **Maintainability:** The codebase shall be modularised into four distinct Docker containers — AI, Cyber, Blockchain, and Web — with defined API contracts between them.
- **Portability:** The software shall be OS-agnostic and capable of running on Windows (via WSL2) or native Linux without code modification.

6. Other Requirements

6.1 Logical Database Requirements

The table below defines the TimescaleDB hypertable schema for off-chain telemetry storage:

Column Name	Data Type	Constraints / Notes
`time`	`TIMESTAMPTZ`	Primary Key (Composite) — partitioned by time
`sensor_id`	`INT`	Primary Key (Composite) — identifies the cell node
`reading_type`	`VARCHAR(32)`	Enum: 'voltage', 'current', 'temperature', 'entropy'
`value`	`FLOAT8`	NOT NULL — the measured or inferred numeric reading
`is_anomalous`	`BOOLEAN`	DEFAULT false — set to true by IDS on spoofing detection

6.2 Regulatory Compliance

- The Blockchain schema shall structurally align with the data ontologies proposed by the Global Battery Alliance (GBA) for the 2027 EU Digital Battery Passport mandates (EU Battery Regulation 2023/1542).

6.3 Risk Management (FMEA)

Failure Mode	Root Cause	Mitigation Strategy
GNN Model Drift	Battery ages differently than the PyBaMM training distribution.	Periodic re-calibration of the GNN against sparse physical sensor ground truth.
IDS False Positive	Extreme but physically valid driving events (e.g., rapid charge pulse).	Cross-reference IDS alerts with strict PyBaMM thermodynamic boundary checks before alerting.

Appendix A: Glossary

- **Thermal Runaway:** A dangerous chain reaction where a battery cell overheats uncontrollably, potentially causing fire or explosion.
- **Spoofing:** A cyberattack in which malicious data is injected into the CAN bus to appear as legitimate sensor readings.
- **GNN (Graph Neural Network):** A neural network architecture that operates on graph-structured data, modelling spatial dependencies between battery cell nodes.
- **RAG (Retrieval-Augmented Generation):** An LLM technique that grounds responses in retrieved documents from a vector database to prevent hallucinations.
- **Battery Passport:** A digital, tamper-proof record of a battery's manufacturing, usage, and health history, mandated by the EU Battery Regulation.

Appendix B: Analysis Models

B.1 System Sequence Diagram

The sequence diagram below illustrates the end-to-end data flow from CAN bus ingestion through AI inference, blockchain anchoring, and dashboard rendering.

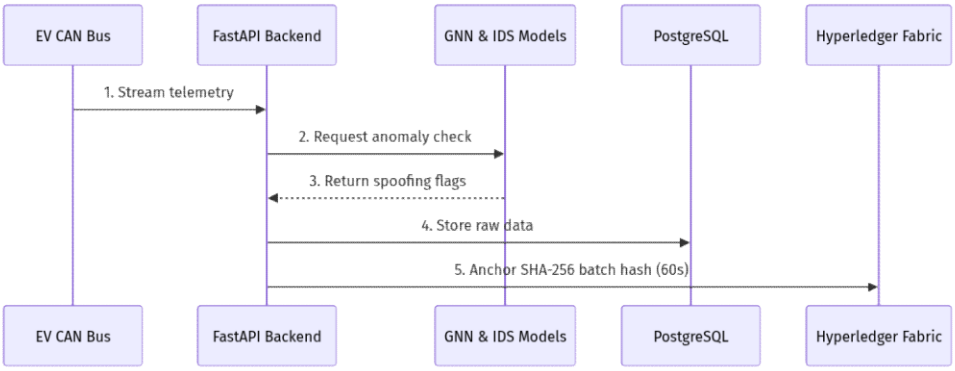


Figure B.1 — TrustSense-BMS System Sequence Diagram

B.2 Entity Relationship Diagram (ERD)

The ERD below defines the logical relationships between the core data entities: telemetry readings, anomaly events, and blockchain passport records.

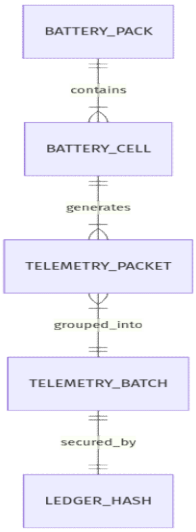


Figure B.2 — TrustSense-BMS Entity Relationship Diagram

Appendix C: Open Issues List

The following items remain to be determined (TBD) pending hardware procurement and field testing:

- `TBD-1:` Final selection of the local LLM weight size — Llama 3 8B vs. 70B — subject to VRAM profiling on the Jetson Orin Nano.
- `TBD-2:` Exact mapping of CAN IDs to specific battery cell indices for the physical testbench procured by VG Marinetech.